WIRELESS COMMUNICATION NETWORKS AND SYSTEMS

LAB 01

Fundamentals of Signal Transmission

Python simulation

Learning Objectives

At the end of this lab, students will be able to:

- Generate and visualize a simple signal (sine wave).
- Understand the effects of **attenuation**, **noise**, and **delay** on transmission.
- Use the **FFT (Fast Fourier Transform)** to analyze a signal in the frequency domain.
- Relate these effects to real-world wireless systems (Wi-Fi, IoT, satellite).

Theoretical Background

A signal is a variation over time (e.g., an electrical wave). When it passes through a **channel** (air, cable, fiber), it may experience:

- **Attenuation** – amplitude decreases (e.g., power loss over distance).
- **Noise** – interference is added (e.g., thermal noise, radio interference).
- **Delay** – it arrives with a temporal shift (e.g., satellite communications).

In practice, a channel is often a combination of these effects.

Required Tools

- Python 3 with `numpy` and `matplotlib`.

Lab Instructions

Step 1 – Generate a sine wave

A sine wave is defined as:

$$s(t) = A \cdot \sin(2\pi f t)$$

```python
import numpy as np
import matplotlib.pyplot as plt

fs = 10000      # sampling frequency (Hz)
f = 1000        # signal frequency (Hz)
```

Copyright © WCNS Team 2026/2027

```
A = 1           # amplitude
t = np.linspace(0, 0.005, int(fs*0.005))  # 5 ms

signal = A * np.sin(2*np.pi*f*t)

plt.figure(figsize=(10,4))
plt.plot(t, signal)
plt.title("Sine Wave (1 kHz)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

Task: Change the frequency to 500 Hz and 2000 Hz. What happens?

Step 2 – Frequency Analysis (FFT)

The FFT decomposes a signal into its frequencies.

```
fft_signal = np.fft.fft(signal)
freqs = np.fft.fftfreq(len(signal), 1/fs)

plt.figure(figsize=(10,4))
plt.plot(freqs[:len(freqs)//2], np.abs(fft_signal[:len(freqs)//2]))
plt.title("Signal Spectrum")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

Question: Identify the dominant frequency in the spectrum.

Step 3 – Attenuation

We simulate power loss by scaling the signal.

```
attenuated = 0.5 * signal

plt.figure(figsize=(10,4))
plt.plot(t, signal, label="Original")
plt.plot(t, attenuated, label="Attenuated")
plt.legend()
plt.title("Effect of Attenuation")
plt.grid(True)
plt.show()
```

Question: Why is attenuation problematic for Wi-Fi?

Step 4 – Add Noise

We add Gaussian noise.

Copyright © WCNS Team 2026/2027

```
sigma = 0.3
noise = np.random.normal(0, sigma, len(t))
noisy_signal = signal + noise

plt.figure(figsize=(10,4))
plt.plot(t, signal, label="Original")
plt.plot(t, noisy_signal, label="Noisy")
plt.legend()
plt.title("Effect of Noise")
plt.grid(True)
plt.show()
```

Task: Vary σ (0.1, 0.5, 1.0) and observe. **Question:** Why can the FFT still recover the main frequency under noise?

Step 5 – Delay

We introduce a time shift.

```
delay = 200
delayed_signal = np.concatenate((np.zeros(delay), signal[:-delay]))

plt.figure(figsize=(10,4))
plt.plot(t, signal, label="Original")
plt.plot(t, delayed_signal, label="Delayed")
plt.legend()
plt.title("Effect of Delay")
plt.grid(True)
plt.show()
```

Question: Give a real example where delay is significant.

Step 6 – Combined Channel

We combine attenuation, noise, and delay.

```
attenuation = 0.6
sigma = 0.3
delay = 100

modified = attenuation * signal
modified += np.random.normal(0, sigma, len(t))
modified = np.concatenate((np.zeros(delay), modified[:-delay]))

plt.figure(figsize=(10,4))
plt.plot(t, signal, label="Original")
plt.plot(t, modified, label="After Channel")
plt.legend()
plt.title("Signal through a Channel")
plt.grid(True)
plt.show()
```

Question: Compare the original and received signals.

Copyright © WCNS Team 2026/2027

Step 7 – Square Wave Variant

```
square_wave = np.sign(np.sin(2*np.pi*f*t))

plt.figure(figsize=(10,4))
plt.plot(t[:200], square_wave[:200])
plt.title("Square Wave (1 kHz)")
plt.grid(True)
plt.show()
```

Question: Why does a square wave require more bandwidth than a sine wave?

Copyright © WCNS Team 2026/2027